Systematic Black-Box Testing of an E-Commerce Pricing Module

Individual lab - Andrei Mihai Visalon
TSS (Testarea Sistemelor Software) - FMI, Master IS, 2026-2027

The complete end-to-end documentation: what is tested, why, and what happens - written to be read alongside the slide deck.

How to read this document

This document and the slide deck (TSS_Lab_Presentation.pptx) are one package built to serve both audiences at once: the deck is what the audience sees; this document is what you study so that every slide's claim is something you can explain, defend and demonstrate. Section order mirrors the presentation arc. Every number in this document is either a constant of the delivered artifacts or was computed live from the module under test (pricing.py) - nothing is invented.

- The folder also contains the core artifacts: SPEC.md (the specification), pricing.py (the module under test), README.md and REPORT.md (the delivered lab), and docs/ (the derivation documents).
- Live demo commands are collected in Section 5.

Plain-language guide: every term spelled out, and why it is needed

This section is a reference you can read aloud. It assumes no background in software testing. Every abbreviation used in the lab is written out in full, explained in everyday words, tied to the question it answers, and given the exact result it produced in this lab, with a concrete example from the pricing module wherever one helps. Read the quick-reference table first; every row then has its own subsection below with the full explanation, in the same order.

First, a short picture of what is being tested, so that every term has something to hang on. The module under test is a shop billing calculator. Give it a cart and it works out the subtotal, takes off any discount, adds shipping, adds an optional extra fee, and adds VAT (Value Added Tax - the tax added to the price of goods, called TVA in Romania). It is made of four functions: quantize_money (rounds an amount of money to whole cents), compute_fee (works out the optional fee), compute_vat (works out the VAT) and compute_totals (runs the whole bill from start to finish by calling the other three). Next to them sits the PricingBreakdown record, which is simply the itemised receipt that compute_totals hands back. The whole module, pricing.py, is 57 statements - think of them as 57 executable lines of code - with 22 branches, which are the true-side and false-side outcomes of its 11 if-decisions.

Quick reference - one line per term; the full explanation of each follows below.

Term	Full name	In one line
SPEC	Specification	The written rulebook that says what the correct answer is for any input - behaviour only, never the code.
Black-box / white-box	Black-box testing / white-box (glass-box) testing	Black-box tests from the outside, by behaviour; white-box looks inside, at the structure of the code.
EP	Equivalence Partitioning	Sort inputs into groups that should behave the same, and test one member of each group.
BVA	Boundary Value Analysis	Test exactly at the edges of each group - zero, the limit, one step past - where mistakes cluster.
CEG	Cause-Effect Graphing	List the on/off switches and the result each combination must give, as a decision table; one test per column.
Line / branch coverage	Line (statement) coverage / branch coverage	Line: was every line run by some test? Branch: was every decision taken both ways?
CFG	Control-Flow Graph	A map of the code: each box is a straight-line chunk, each arrow a possible jump.
V(G)	Cyclomatic complexity of the graph G (McCabe's measure)	How many independent routes (a basis set) run through the code - 19 here; every other route is a combination of them.
MC/DC	Modified Condition/Decision Coverage	Prove that every single yes/no condition can, on its own, change the decision.
Mutant / order-1 mutant	Mutant; first-order (order-1) mutant	A copy of the program with one small deliberate fault; order-1 means exactly one change.
Equivalent mutant	Equivalent mutant	A planted change that makes no difference to any output, so no test can ever catch it.
Killed / surviving mutant	Killed mutant / surviving (live) mutant	Killed: some test notices the fault. Surviving: the tests that were run do not.
Mutation score	Mutation testing; mutation score	Plant many faults automatically and count how many the tests catch: killed / (killed + survived).
mutmut / cosmic-ray	Two Python mutation-testing tools	Two independent fault-planting tools with different kinds of change, so different totals and scores.
Hypothesis	Property-based testing with the Hypothesis library	State a rule that must hold for every input and let the tool hunt for an input that breaks it.

Term 1 - Specification (SPEC)

- **Spelled out:** SPEC is short for Specification. In this package it is the file SPEC.md.
- **In everyday words:** The specification is the rulebook. For any bill you could give the calculator, it says what the right answer is - for example, that the fee is worked out on the price after the discount, not before it. It is written in black-box terms: it only talks about what goes in and what must come out, and never about how the code is written inside.
- **Why the lab needs it:** A test is a question with a known right answer. Without a written rulebook there is nothing to compare the program's output against, and 'correct' would quietly come to

mean 'whatever the code happens to do'. Every EP, BVA and CEG test in this lab is derived from the SPEC, not from the code, so those tests check what the program SHOULD do rather than copying what it already does. MC/DC is the one structural exception: its list of conditions is read from the code itself (see Term 2). (Section 1 explains openly that the program existed before the lab, and how that is handled.)

- **The result in this lab:** SPEC.md describes the four functions (quantize_money, compute_fee, compute_vat, compute_totals) and the PricingBreakdown record. The EP, BVA and CEG suites - 52, 129 and 17 tests - and the 8 Hypothesis properties are derived from it; the 14 MC/DC tests combine the SPEC's MC/DC rules with the list of conditions read from pricing.py. Together the four example-based suites hold 212 tests.
- **Example from the pricing module:** The SPEC says that when the fee is switched off, the fee is 0.00. A test then switches the fee off and checks that 0.00 comes back. The test came from that sentence of the SPEC, not from reading the line of code that does it.

Term 2 - Black-box testing vs white-box testing

- **Spelled out:** Black-box testing (also called behavioural or specification-based testing) and white-box testing (also called glass-box or structural testing).
- **In everyday words:** Imagine the program is a sealed box. In black-box testing you are not allowed to open it: you put inputs in, look at what comes out, and compare that with the rulebook. In white-box testing the box is made of glass: you look at the code inside and ask whether your tests actually travelled through every part of it.
- **Why the lab needs both:** They answer different questions. Black-box asks 'does it do the right thing?'. White-box asks 'did my tests reach every corner of the code?'. Black-box testing alone can leave parts of the code that no test ever runs; white-box measurement alone can show every line was run without ever checking an answer against the rulebook. So the lab DESIGNS its tests black-box, from the SPEC, and then MEASURES them white-box, with coverage and the control-flow graph.
- **The result in this lab:** EP, BVA and CEG are black-box design techniques. MC/DC is a structural (white-box) coverage criterion, and it is used as one here: its list of conditions (C1-C14 in docs/mcdc.md) is read from the if-statements of pricing.py, and docs/mcdc.md records that its expected values were worked out by hand by tracing the code paths. It is the one technique in this lab whose tests are not derived from the SPEC alone. Phase 4 - line and branch coverage, the CFG and V(G) - is the white-box measurement. Phase 6, mutation testing, plants faults in the code and uses the black-box tests to try to catch them.
- **Example from the pricing module:** A black-box question: 'with VAT switched off, is the VAT 0.00?'. A white-box question: 'did any test ever run line 78 - the shortcut inside compute_vat that returns 0.00 when VAT is switched off?'. As the coverage subsection below shows, the first can be answered yes while the second is still no.

Term 3 - EP: Equivalence Partitioning

- **Spelled out:** EP stands for Equivalence Partitioning (also called equivalence class partitioning).
- **In everyday words:** 'Equivalence' means 'behaves the same'; 'partitioning' means 'splitting into groups'. You split all possible inputs into groups whose members the rulebook treats in exactly the

same way, and you test just one member of each group - because if one member is handled correctly, the rulebook gives no reason to expect the others in the same group to be handled differently. Each group is called an equivalence class, or a partition.

- **Why the lab needs it:** There are infinitely many possible bills, so testing them all is impossible. EP gives the smallest set of tests that still touches every distinct rule in the SPEC at least once. It is the first, broad sweep, and it answers the question: 'does every kind of behaviour the rulebook describes actually happen?'
- **The result in this lab:** 52 tests (tests/test_ep.py). Each test's description names the group it represents, and the groups are listed in docs/ep.md. Run on its own, this suite already reaches 100.00% line coverage (57/57) and 100.00% branch coverage (22/22).
- **Example from the pricing module:** Every input in which the fee is switched off behaves identically - the fee is 0.00 whatever the subtotal, fee type or fee value - so one test represents that whole group.

Term 4 - BVA: Boundary Value Analysis

- **Spelled out:** BVA stands for Boundary Value Analysis.
- **In everyday words:** A boundary is the edge of a group - the exact point where one rule stops and the next one starts. BVA tests right at each edge: the value exactly on it, and one small step either side of it. Experience shows that mistakes cluster at edges: a programmer writes 'less than' where the rule says 'less than or equal to', or rounds the wrong way when an amount sits exactly halfway between two cents.
- **Why the lab needs it:** EP proves that each kind of behaviour exists, but it checks only one point somewhere inside each group. Most real faults sit at the edges, not in the middle. BVA answers the question: 'does the program switch from one rule to the next at exactly the right place?'
- **The result in this lab:** 129 tests (tests/test_bva.py) - the largest suite, because money faults concentrate at edges - with the boundary tables in docs/bva.md. Typical edges probed: zero, negative amounts, the zero fee value (the free-tier edge), and representative halfway 'tie' amounts, such as 0.005 and 2.345, under every rounding mode. Run on its own it reaches 96.49% line coverage (55/57) and 90.91% branch coverage (20/22); the coverage subsection at the end of this guide explains why that is the expected result for this technique.
- **Example from the pricing module:** A rounding tie at 2.345. The amount sits exactly halfway between 2.34 and 2.35, and the four rounding modes do not all agree on which way it should go (section 2.1 lists the four answers). Only a test placed exactly on the tie can show whether each mode rounds the way the SPEC says.

Term 5 - CEG: Cause-Effect Graphing

- **Spelled out:** CEG stands for Cause-Effect Graphing. The technique draws a cause-effect graph and then turns it into a decision table.
- **In everyday words:** A 'cause' is an input condition that is either on or off, like a light switch. An 'effect' is a result the program must produce. You draw which switches lead to which results, then turn the drawing into a decision table: one column per meaningful combination of switches, each column saying what the bill must look like. Every column becomes exactly one test. Combinations that cannot make any visible difference are merged, so no tests are wasted on them.

- **Why the lab needs it:** Some faults appear only when two switches interact - for example, 'charge VAT on the fee' matters only if VAT is switched on and there is a fee at all. Testing switches one at a time misses such interactions; testing every possible combination blindly wastes effort. CEG answers the question: 'for every combination of switches that can make a difference, does the program give the result the SPEC requires?'
- **The result in this lab:** The main switches are: fee on/off, VAT on/off, apply VAT to shipping (yes/no) and apply VAT to the fee (yes/no); the full list in docs/ceg.md has 14 causes: these four plus fee value > 0, fee type (flat or percent), VAT rate > 0, composed VAT base > 0, override present, override > 0, discount > 0, shipping > 0, and two derived sign conditions on the subtotal. The decision table gives 17 tests (tests/test_ceg.py). Run on its own, the suite reaches 98.25% line coverage (56/57) and 95.45% branch coverage (21/22).
- **Example from the pricing module:** When VAT is switched off, the two switches 'apply VAT to shipping' and 'apply VAT to the fee' can make no visible difference - there is no VAT to apply them to - so the table merges all of those combinations into a single column instead of testing each one.

Term 6 - Line coverage and branch coverage

- **Spelled out:** Line coverage (also called statement coverage) and branch coverage. Both are measured here with the coverage tool for pytest (pytest-cov) and written as covered / total.
- **In everyday words:** Coverage is a measuring tape for how much of the code your tests touched. Line coverage asks, for every line of code: did at least one test make the computer run it? Branch coverage is stricter. Every decision ('if this, do that, otherwise carry on') has two sides, the true side and the false side, and branch coverage asks whether the tests took BOTH sides of EVERY decision. You can run every line and still never take one side of a decision - for example when the 'otherwise' side has no lines of its own - which is why branch coverage is the stronger of the two.
- **Why the lab needs it:** It is the first white-box check on the black-box tests. A piece of code that no test ever runs is a piece of code whose bugs nobody could ever see. Coverage answers the question: 'is there any part of the program my tests never even visited?'
- **The result in this lab:** pricing.py has 57 statements and 22 branches. Run together, the four example-based suites (212 tests) reach 100.00% line coverage (57/57) and 100.00% branch coverage (22/22); with the 8 Hypothesis properties added (220 tests) the figures stay at 100.00% and 100.00%. The delivered gate runs the full suite and refuses to pass below 100%. Each suite's own figures, run separately, are in the table at the end of this guide.
- **Example from the pricing module:** In compute_vat, 'if apply_to_fee: base += fee' is a decision. A test with apply-to-fee switched on takes the true side and runs line 85; a test with it switched off takes the false side and skips it. Branch coverage is satisfied only when both have happened.

Term 7 - CFG: Control-Flow Graph

- **Spelled out:** CFG stands for Control-Flow Graph. 'Control flow' is the order in which the computer moves through the code; a 'graph' here means a drawing of boxes joined by arrows.
- **In everyday words:** The CFG is a map of the code, much like a metro map. Each box (called a node) is a chunk of code that always runs straight through from top to bottom, with no decision inside it. Each arrow (called an edge) is a possible jump from one box to the next; wherever there

is a decision, two arrows leave the box - one for the true side and one for the false side. Following the arrows from a start box to an end box traces one possible run of the program.

- **Why the lab needs it:** The map turns 'how complicated is this code, and in how many different ways can it run?' from an opinion into something that can be counted. It is the input to V(G) (the next term), which says how many distinct paths a structured test plan must cover, and it makes it possible to check, path by path, that the tests really drive each one.
- **The result in this lab:** 51 boxes (nodes) and 62 control arrows (edges). There are also 11 dashed 'call' arrows, which show where one function calls another (for example compute_totals calling compute_fee); they are drawn for readability but are NOT counted as control edges. The graph's nodes and edges were derived by hand from pricing.py and encoded in docs/gen_cfg.py, which writes docs/cfg.drawio (draw.io format, so it opens in draw.io) and docs/cfg.svg, then runs 69 checks on the graph and prints ALL CHECKS PASSED.
- **Example from the pricing module:** In compute_fee, the box that asks 'is the fee switched off?' has two arrows leaving it: the true arrow goes to a box that returns 0.00 straight away, and the false arrow goes on to the next question, 'is the fee value zero or less?'.

Term 8 - V(G): cyclomatic complexity (McCabe's measure)

- **Spelled out:** V(G), read 'V of G', is the cyclomatic complexity of the graph G - the control-flow graph from the previous term. It is also called McCabe's measure, after Thomas McCabe, who defined it. The word 'cyclomatic' comes from graph theory, where the closely related cyclomatic number counts the independent cycles of a graph; McCabe's value equals it once each function's exit is joined back to its entry (pricing.py itself has no loops).
- **In everyday words:** Think of the decisions in the code as forks in a road. V(G) is a single number that answers: 'how many genuinely different routes run through this code?'. A route counts as genuinely new only if it takes at least one turn that cannot be obtained by combining the routes you already have.
- **The formula:** V(G) = E - N + 2P, where E is the number of edges (control arrows), N is the number of nodes (boxes), and P is the number of separate, unconnected pieces of the graph. Here E = 62 control edges, N = 51 nodes, and P = 4, because the module has four independent functions and each function's map is its own separate piece. So V(G) = 62 - 51 + 2 x 4 = 19. The 11 dashed call arrows are left out, because McCabe's definition counts only the control flow inside the code, not the calls between functions.
- **What 19 means in practice:** 19 is the size of the basis set: the smallest collection of routes from which every other route through the code can be assembled. Such routes are called linearly independent - none of them can be made by combining the others. If the tests follow a complete basis set, every branch is guaranteed to have been taken at least once, which is why 19 is the size of the path set a structured (basis-path) test plan works from. Here only 18 of the 19 can be executed: the 19th (T-P8 in compute_totals) needs 'total < 0' to be true, which no input can produce - the same reason line 148 carries '# pragma: no cover' - so it is documented as infeasible instead of tested. A precise footnote, in case an examiner asks: plain branch coverage can sometimes be reached with fewer runs than V(G); V(G) is the exact size of a basis set and an upper bound on the runs that branch coverage needs.

- **Why the lab needs it:** It turns 'have I tested enough different ways through the code?' into a number that can be checked, and it is a standard yardstick for how hard a piece of code is to test. Do not confuse 19 with the 22 branches the coverage tool counts: the 22 are decision outcomes (the true and false sides) that the tool tracks, while 19 counts independent routes - they measure different things and are not expected to be equal. (The coverage tool tracks the 11 if-statements but not the three one-line conditional expressions at lines 108, 115 and 128, and the pragma excludes the guard at line 148; the CFG counts all 15 decision points, so V(G) = 15 + 4 = 19.)
- **The result in this lab:** V(G) = 19, from 62 control edges, 51 nodes and 4 functions. docs/gen_cfg.py checks this as part of its 69 checks, and docs/cfg-analysis.md lists the basis paths. Closing each basis path with its function's EXIT-to-ENTER return edge turns it into a circuit, so the 19 basis paths are also the 19 linearly independent circuits the assignment asks for, and together they cover every arc; that closed graph is the 'circuit derived from the graph', and its cyclomatic number (62 + 4) - 51 + 4 = 19 equals V(G) (docs/cfg-analysis.md, section 4.1).
- **Example from the pricing module:** compute_fee on its own has these genuinely different routes: fee switched off, giving 0.00; fee on but its value zero or less, giving 0.00; a percent fee, giving that percentage of the taxable subtotal; and a flat fee, giving the fee value itself. Each is a separate route that a basis set must include.

Term 9 - MC/DC: Modified Condition/Decision Coverage

- **Spelled out:** MC/DC stands for Modified Condition/Decision Coverage.
- **In everyday words:** A decision in code is often built from several smaller yes/no questions joined by AND or OR - for example 'is VAT switched on AND is the rate above zero AND is there anything to charge it on?' (in pricing.py these are three separate single-condition ifs, so each MC/DC pair in this lab flips one single-condition decision). Each small question is called a condition. MC/DC demands, for every single condition, a pair of test runs that are identical except for that one condition, where flipping that one condition flips the final decision. That pair proves the condition genuinely matters on its own: if it were written wrongly, ignored, or swapped with another, one of the two runs would expose it.
- **Why the lab needs it:** Branch coverage only checks that the whole decision came out true once and false once. With 'A AND B' you can get both outcomes without B ever being the reason - B could be broken and branch coverage would not notice. MC/DC closes that gap. It answers the question: 'does every individual condition actually affect the outcome the way the SPEC says?'. It is the coverage standard required for the most safety-critical (Level A) civil-aviation software under DO-178C, the guideline used to approve software that flies on aircraft.
- **The result in this lab:** 14 reachable conditions -> 14 tests (tests/test_mcdc.py; the 15th condition, the total < 0 guard at line 148, is unreachable and has no pair), with the truth tables and the independence pairs in docs/mcdc.md. Run on its own, the suite reaches 100.00% line coverage (57/57) and 100.00% branch coverage (22/22).
- **Example from the pricing module:** An MC/DC pair for the condition 'VAT is switched on': two runs with the same cart and the same positive rate, one with VAT on and one with VAT off. The VAT flips between a real amount and 0.00, so 'switched on' provably matters by itself.

Term 10 - Mutant and order-1 mutant

- **Spelled out:** A mutant is a mutated - deliberately altered - copy of the program. Order-1 (also called first-order) means the copy differs from the original by exactly one change; a higher-order mutant would carry two or more changes at once.
- **In everyday words:** A mutant is a copy of the program with one tiny fault planted on purpose: a '<' changed to '<=', a '==' changed to '!=', a 0 changed to another number. It is a fire drill for the tests: you start a small, controlled fire and see whether the alarm goes off. Keeping it to exactly one change (order-1) means that when a test catches it, you know precisely which fault was caught.
- **Why the lab needs it:** Passing tests prove only that the program agrees with the tests. Mutants measure the tests themselves. They answer the question: 'if the program were wrong in this specific, small way, would my tests notice?'
- **The result in this lab:** Three hand-built order-1 mutants, each in its own file in mutants/: an equivalent one (mutant_equivalent.py), a killed one (mutant_killed.py) and a surviving one (mutant_surviving.py); mutants/run_demo.py runs them side by side. On top of these, two tools generate many more mutants automatically, one change each (Terms 13 and 14).
- **Example from the pricing module:** mutant_killed.py is pricing.py with exactly one change, inside compute_fee: fee_type == "percent" became fee_type != "percent". Everything else is identical.

Term 11 - Equivalent mutant (assignment point 5)

- **Spelled out:** An equivalent mutant is a mutant that is behaviourally equivalent to the original program - indistinguishable from it by any input.
- **In everyday words:** Sometimes a planted change looks like a fault but turns out to make no difference at all: for every possible input, the altered program gives exactly the same output as the original. It is like editing a rule that can never be triggered: the wording is different, but nothing anyone does can ever reveal the difference.
- **Why the lab needs it:** The assignment asks for one (point 5), and its lesson is fundamental: no test can catch a change that changes nothing. That is why a perfect mutation score - every mutant killed - is impossible in principle whenever equivalent mutants exist, and why surviving mutants must be examined one by one: some reveal a weak test, some are equivalent and reveal nothing about the tests at all.
- **The result in this lab:** mutant_equivalent.py changes the final safety clamp in compute_totals from 'if total < 0' to 'if total <= 0'. The two versions are proven to give identical results on every input (under Python's default decimal context, which the module never changes).
- **Example from the pricing module:** The only total on which '< 0' and '<= 0' disagree is a total of exactly zero. There, the original leaves the total at 0.00 and the mutant resets it to 0.00 - the same value. A negative total can never occur in the first place, because every amount added into it is already zero or positive (the code itself marks that guard as unreachable by construction). So no input, from any test, can tell the two programs apart.

Term 12 - Killed vs surviving mutant (assignment point 6)

- **Spelled out:** A killed mutant is one that at least one test detects - that test fails when it is run against the mutant. A surviving mutant (also called a live mutant) is one against which all the tests that were run still pass.

- **In everyday words:** You run the tests against the copy with the planted fault. If any test goes red (fails), the fault was caught: the mutant is 'killed', which is good news about the tests. If every test stays green, the fault slipped through: the mutant 'survives', and it points to exactly the kind of mistake those tests would miss.
- **Why the lab needs it:** Showing one of each (point 6) proves two things at once: that the tests are strong enough to catch a real, plausible fault, and exactly where a weaker test would let one through. It answers the question: 'what does catching a fault look like, and what does missing one look like?'
- **The killed mutant in this lab:** mutant_killed.py changes fee_type == "percent" to fee_type != "percent". That swaps the meaning of the two fee types: a percentage fee would be charged as if it were a fixed amount, and a fixed fee as if it were a percentage. Tests whose fee is computed on a taxable subtotal other than 100 can see the swap: 20 of the 212 tests fail, including the chosen test, whose taxable subtotal is 200, so the fault is caught - the mutant is killed. At a taxable subtotal of exactly 100 the swap is invisible, because a percentage of 100 equals the value itself.
- **The surviving mutant in this lab:** mutant_surviving.py changes fee_value <= 0 to fee_value <= 4. Now any fee value of 4 or less - not just zero or less - is wrongly treated as 'no fee'. The demo's chosen test is the MC/DC test for the fee TYPE (test_mcdc_c03_compute_fee_type_percent), which holds the fee value fixed at 5; 5 is above 4, so the original and the mutant give the same answer and that test stays green: the mutant survives that test. The full suite, however, kills it with 10 failures: 8 boundary-value tests that use small positive fee values of 4 or less (for example 0.005, or the 2.345 rounding tie), plus the two MC/DC tests written for the fee rules themselves (C1 and C2), which use a fee value of 3. The mutant wrongly turns all of those fees into 0.00, so those tests fail.
- **The lesson:** The surviving case is not a failure of the lab - it is the demonstration. A test that holds its other inputs at convenient fixed values (MC/DC deliberately keeps everything except the one condition steady) can miss a fault that a boundary test catches at once. Different techniques see different faults, which is exactly why the lab combines them.

Term 13 - Mutation testing and mutation score (assignment point 7)

- **Spelled out:** Mutation testing is the method; the mutation score is its result: mutation score = killed / (killed + survived), usually written as a percentage.
- **In everyday words:** Instead of planting three faults by hand, you let a tool plant many of them automatically. It walks through the code and, at every place it can, makes one small change - flipping a comparison, swapping one arithmetic operator for another, changing a constant, and so on - producing one mutant per change. It then runs your tests against every mutant and counts how many were caught. The mutation score is simply the share of planted faults that the tests caught: the higher it is, the more of the plausible mistakes the tests would notice.
- **Why the lab needs it:** Coverage can tell you that a line ran; it cannot tell you whether a mistake on that line would be noticed. Mutation testing checks exactly that, mechanically, and answers the question: 'would these tests actually notice if the code were wrong?'. Running each suite on its own against the same mutants also compares the techniques fairly: it shows which technique catches which kinds of fault (point 7).
- **The result in this lab:** mutmut 3.8.0 plants 176 mutants and cosmic-ray 8.7.0 plants 226. Each tool was run against each of the four example-based suites separately; the per-suite scores are

tabled under Term 14, and the explanation of every class of survivor is in docs/mutation-analysis.md.

- **Example from the pricing module:** A tool might change 'if taxable < 0' in compute_totals to 'if taxable <= 0', or turn 'base += shipping' in compute_vat into 'base -= shipping'. Each such change is one mutant, and each one either makes some test fail (killed) or does not (survived).

Term 14 - The two mutation tools (mutmut and cosmic-ray) and why their scores differ

- **Spelled out:** mutmut and cosmic-ray are two independent, open-source mutation-testing tools for Python. Their names are product names, not abbreviations. The kinds of change each tool knows how to make are called its mutation operators.
- **In everyday words:** Think of them as two different inspectors, each with their own checklist of 'small mistakes a programmer could make'. Each checklist item is a mutation operator - a kind of change, such as 'flip a comparison' or 'alter a piece of text'. Because the checklists differ, the two inspectors plant different numbers of mutants in the same code, and each plants some kinds of mutant the other never does.
- **Why the lab needs it:** One tool's score could be an artefact of its particular checklist. Running two independent tools and comparing them shows which conclusions are robust - both tools agree that a technique is weak in some place - and which depend on the tool. Comparing the tools is itself one of the delivered analysis points.
- **The result in this lab:** mutmut 3.8.0 plants 176 mutants; cosmic-ray 8.7.0 plants 226. The totals differ, and therefore the per-suite scores differ, because the tools use different mutation operators. For example, cosmic-ray also mutates the type annotation - the '|' in the signature 'vat_override: Decimal | None = None', which only describes the expected type and does nothing when the program runs - while only mutmut mutates string literals (pieces of text in quotes, such as "percent" or "half_up"); cosmic-ray has no string operator at all. A change to something that does nothing at run time cannot be caught by any test, which is one reason the raw scores of the two tools are not directly comparable. The per-suite scores for both tools are in the table below; the full analysis is in docs/mutation-analysis.md.

Per-suite mutation scores - each suite run on its own against every mutant the tool planted; score = killed / (killed + survived); both tools re-run per suite on 05.11.2026 (docs/mutation-analysis.md):

Tool	Mutants	EP	BVA	CEG	MC/DC
mutmut 3.8.0	176	75.57% (133 killed)	77.84% (137)	66.48% (117)	61.36% (108)
cosmic-ray 8.7.0	226	82.30% (186 killed)	88.05% (199)	80.97% (183)	81.42% (184)

- **Why no suite reaches 100%:** That is expected, not a failure. The survivors include equivalent mutants - changes that make no difference to any output, which no test can ever kill (Term 11) - and changes that no assertion in that particular suite checks, which is exactly the per-technique blind spot the comparison is meant to expose.
- **Why cosmic-ray scores higher:** cosmic-ray scores roughly 7 to 20 points higher than mutmut for every suite, because it plants a different and larger set of mutants (226 against 176) with different

mutation operators. A score only means something relative to the set of mutants it was measured on, so the two tools are compared side by side, never merged into one number.
- **Where the detail is:** docs/mutation-analysis.md breaks the results down operator by operator and explains each class of survivor.

Term 15 - Property-based testing with Hypothesis (Phase 7 bonus)

- **Spelled out:** Property-based testing; Hypothesis is the name of the Python library that does it. A 'property' here means a rule (an invariant) that must be true for every input.
- **In everyday words:** Every earlier technique tests examples that I chose: 'for this cart, the answer must be exactly this'. Property-based testing turns that around. I state a rule that must hold for ANY cart - for example 'the VAT is never negative', or 'rounding an amount that is already rounded changes nothing' - and the tool invents random carts, including odd ones I would never think to write, trying to find one that breaks the rule. If it finds one, it shrinks it: it simplifies the failing cart step by step until it has the smallest, simplest example that still breaks the rule, and reports that.
- **Why the lab needs it:** Hand-chosen examples, however systematic, leave gaps between the chosen points. A property checks a relationship everywhere the tool can reach, so a fault that shows up only at some odd value in between has nowhere to hide. It answers the question: 'is there any input at all - not just the ones I thought of - that breaks a rule the SPEC guarantees?'
- **The result in this lab:** 8 properties (tests/test_properties.py), listed in section 8.1. In the normal run each property is tried on 200 random inputs; in the deep run, switched on by setting SLOW=1 in the environment, each is tried on 10,000. No counterexample was found in either run. Hypothesis also proved its worth along the way: one of the properties was first written wrongly, and the tool's shrunk counterexample showed exactly why (section 8.1, property T2).
- **Example from the pricing module:** Property Q1 - rounding twice equals rounding once: take any amount, round it to cents with quantize_money, then round the result again; the second rounding must change nothing, in all four rounding modes.

Why some single-technique coverage is under 100%, and why that is correct

Assignment point 2 asks for the coverage of each technique's suite to be measured and compared. To compare the techniques fairly, each suite is run ON ITS OWN, as if it were the only test suite in existence. These are the per-suite results:

Suite	Tests	Line coverage	Branch coverage	Missing lines
EP	52	100.00% (57/57)	100.00% (22/22)	none
BVA	129	96.49% (55/57)	90.91% (20/22)	78, 85
CEG	17	98.25% (56/57)	95.45% (21/22)	78
MC/DC	14	100.00% (57/57)	100.00% (22/22)	none
Combined (1-4)	212	100.00% (57/57)	100.00% (22/22)	none
All (+Hypothesis)	220	100.00% (57/57)	100.00% (22/22)	none

Under 100% for a single technique is the FINDING, not a defect. Assignment point 2 asks for each technique to be run on its own precisely so that the techniques can be compared - and the comparison is only informative if it can show where a technique, by itself, falls short.

Both missed lines are in compute_vat. Line 78 is the shortcut that returns 0.00 under 'if not enabled:' - the exit taken when VAT is switched off. Line 85 is 'base += fee' under 'if apply_to_fee:' - the step that adds the fee into the amount VAT is charged on.

Why BVA misses lines 78 and 85: Boundary Value Analysis probes NUMERIC edges - zero, one step past zero, a halfway rounding tie, a 100% rate. An on/off flag has no numeric edge to probe: it is simply on or off. Line 78 is the exit taken only when compute_vat itself is called with VAT switched off, so no boundary probe leads there - that miss is a property of the technique. Line 85 could be reached by a boundary probe that crosses the VAT base's zero boundary through the fee instead of the shipping, but the suite's base-boundary sweep (B12) crosses zero through shipping only and never switches apply-to-fee on, so its 129 tests do not reach it - a design choice within the suite, closed by EP and MC/DC in the combined run.

Why CEG misses line 78: Cause-Effect Graphing does include the VAT-switched-off case - but it tests that case through the full calculation, compute_totals, the way a real bill is produced, not as a direct call to compute_vat. compute_totals checks 'is VAT switched on?' itself and, when it is off, sets the VAT to 0.00 without ever calling compute_vat. The shortcut inside compute_vat (line 78) is therefore reached only by calling compute_vat directly with VAT switched off, which the CEG tests do not do. The VAT-off behaviour IS tested; that particular line is simply not on CEG's route.

The whole point of running the suites separately is to SEE these blind spots. Run together, the techniques cover each other: the Combined suite (techniques 1-4, 212 tests) and the All suite (220 tests, with the Hypothesis properties) both reach 100.00% line and 100.00% branch coverage. The delivered gate runs the full suite and enforces 100%: if coverage ever dropped below that, the run would fail.

What you will see on screen during a live run: when a single suite is run on its own, the coverage report that pytest-cov prints in the terminal shows ONE combined "Cover" figure, which merges lines and branches into a single number. For BVA it shows (55 + 20) / (57 + 22) = 75/79 = 94.94%; for CEG it shows (56 + 21) / (57 + 22) = 77/79 = 97.47%. This document reports line coverage and branch coverage separately instead (for BVA: 96.49% line, 90.91% branch). Both are correct - they measure different things: the terminal figure pools covered lines and covered branches together, while the table above keeps them apart. The separate figures come from the coverage comparison run (run_lab.ps1 coverage), which reads coverage.json. Run a single suite with --cov-fail-under=0 (as run_lab.ps1 bva does); otherwise the 100% gate in pyproject.toml prints 'FAIL Required test coverage of 100.0% not reached' and exits with code 1 even though every test passed - that FAIL is the gate, not a test failure.

The big picture: 100% coverage proves only that every line RAN during testing - it does not prove the answers were RIGHT, because a test can run a line and check nothing. That is why this lab goes beyond coverage to mutation testing, which asks the harder question: would these tests actually notice if the code were wrong? The combined suite reaches 100% coverage AND kills the two hand-built faults - the killed and the surviving mutant alike. The third hand-built mutant, the proven-equivalent one, is not a fault at all: it changes nothing, so no test could ever catch it. The tool-generated mutants are a separate measurement, reported per suite under Term 14. Coverage still does not mean every fault is caught: cosmic-ray's mutant 'vat_rate_percent <= 0 -> <= -1' (a rate of -0.5 then gives a VAT of -0.50

instead of 0.00) passes all 212 example-based tests, and only the Hypothesis property T3 - VAT is never negative - catches it (measured 05.11.2026).

1. The module under test - and why this target

The program is a pure-stdlib (decimal + dataclasses) e-commerce pricing module of four public functions and one result record. It is a behavior-identical port of production code from my own repository (momentstudio), which gives the lab a realistic target: these are rules a real store runs, not a toy.

Why this target is a good test subject:

- Money rounding is genuinely tricky - four configurable rounding modes do not always agree on the same amount, and the two 'half' modes disagree only on an exact tie such as 2.345 - a classic real-world fault site.
- Flags interact: whether VAT applies to shipping or to the fee only matters when VAT is enabled at all - natural cause-effect structure.
- Composition: compute_totals chains every other function, so structural coverage of the composition stress-tests each part in context.
- It is small enough to reach 100% coverage honestly, yet rich enough that mutation testing still finds surviving faults - the sweet spot for a lab about method.

Provenance, stated plainly (as in the delivered README): the program pre-dates the lab; the specification was therefore derived by observing the module's behavior. This is disclosed, and mitigated three ways: the disclosure itself; the spec is written purely in black-box terms (no implementation detail); and the EP, BVA and CEG suites are derived from the SPEC, never from the code (MC/DC, a structural criterion, takes its list of conditions from the code, as docs/mcdc.md states).

2. The functions - specified, and why each is tested

This is the heart of 'which functions did I test and why'. Each function gets: what it does (from SPEC.md), the rule an implementation could get wrong, and a worked example computed from the real module.

2.1 quantize_money(value, rounding) - the money rounding primitive

- SPEC: round any decimal amount to exactly 2 decimal places, under one of four modes: half_up, half_even, up, down.
- The fault site - ties: 2.345 becomes 2.35 (half_up), 2.34 (half_even), 2.35 (up), 2.34 (down). All four are correct answers to different questions; confusing them is a real money bug.
- Why tested first and hardest: every other function calls it, so its boundary behavior multiplies through the whole module.

2.2 compute_fee(*, taxable_subtotal, enabled, fee_type, fee_value, rounding='half_up') - the fee

- SPEC: if disabled or the fee value is <= 0, the fee is 0.00; if type is flat the fee is the value itself; if percent it is value% of the taxable subtotal. Result is quantized to 2 dp.
- Worked examples: flat 5 on 200 -> 5.00; percent 10% of 200 -> 20.00. Free-tier rule: fee_value 0 or enabled=False -> 0.00.
- Fault sites: the disabled/zero short-circuit, the type switch, and percent computation rounding (a percentage of a subtotal can land exactly on a tie).

2.3 compute_vat(*, taxable_subtotal, shipping, fee, enabled, vat_rate_percent, apply_to_shipping, apply_to_fee, rounding='half_up') - the VAT

- SPEC: if disabled, or if the rate is <= 0 -> 0.00; otherwise the base is the taxable subtotal plus shipping and/or the fee when their flags are on, and the VAT is rate_percent% of that base, quantized (0.00 when the base is <= 0). The VAT override is not an input of compute_vat: compute_totals handles it (rule T6), and an override REPLACES the rate-based VAT entirely (not added, not compared).
- Fault sites: apply_to_shipping / apply_to_fee flag interactions, composed base <= 0 when a negative input offsets a positive one, rate <= 0 boundaries, and in compute_totals (rule T6) the vat_override branch where 'replaces' is easy to misimplement as 'adds' or 'max of'.
- Why tested: VAT is the rule with an either/or identity - the perfect target for MC/DC condition-independence later.

2.4 compute_totals(...) - the composition

- SPEC, in order: apply discount to the subtotal -> taxable_subtotal; compute the fee on the TAXABLE subtotal (not the original!); compute VAT on the taxable subtotal plus - only if the flags say so - shipping and/or fee; sum to the total. Every intermediate is quantized at its own step.
- The canonical walk (computed from the real module): subtotal 100, discount 10 -> taxable 90.00; shipping 15.00; fee 10% of taxable -> 9.00; VAT 19% of (taxable + shipping + fee) = 19% of 114 -> 21.66; total 135.66.
- Fault sites: fee on pre-discount subtotal (classic), VAT base including parts the flags exclude, quantization order (rounding twice vs once changes cents).

2.5 PricingBreakdown - the record

A dataclass carrying every intermediate (subtotal, discount, taxable_subtotal, shipping, fee, vat, total). Its fields are checked by every compute_totals test: the 17 CEG tests compare the whole record, while the EP, BVA and MC/DC tests assert the fields each rule drives. The whole-record comparison is what makes the CEG composition tests precise about WHERE a fault lands.

3. What happens, end to end - the seven phases

The lab is one story in seven phases; each phase exists because the previous one is insufficient.

Phase 1 - Equivalence Partitioning (52 tests)

For every input, the SPEC's rules partition the domain into classes treated identically: amounts (zero/positive/negative), rates (negative/zero/between 0 and 100/100 and above), flags, fee types, rounding modes. One representative per class. Why first: it is the cheapest complete sweep of distinct behavior - every rule in the spec is touched at least once before boundaries refine anything. The tests name their partitions in leading comments and in their function names, so the suite audits against docs/ep.md's partition table.

Phase 2 - Boundary-Value Analysis (129 tests)

Partitions say what is equal; implementations break where equality ends. Every partition edge becomes tests: 0.005 (the first tie), representative x.xx5 ties under every mode, zero and negatives, the free-tier boundary (fee value 0). The tie family is the star: 2.345 rounds to 2.35 under half_up and up but to 2.34 under half_even and down - four modes, two distinct answers - so the suite probes the tie together with its neighbours (2.344, 2.3449, 2.3451, 2.346), the set that tells all four modes apart; a single wrong constant in the rounding table survives green tests until a boundary test asks exactly these questions. This is the largest suite because money faults concentrate at edges.

Phase 3 - Cause-Effect Graphing (17 tests)

Causes (docs/ceg.md, section 1): the twelve SPEC conditions - fee_enabled, fee_value > 0, fee_type == "percent", vat_enabled, vat_rate_percent > 0, vat_apply_to_shipping, vat_apply_to_fee, composed VAT base > 0, override present, override > 0, discount > 0, shipping > 0 - plus two derived sign conditions on the subtotal. Effects (E1-E10): the fee is 0, flat or percent; the VAT is 0 because VAT is off, 0 because a non-positive override is clamped, equal to a positive override, 0 by the rate or base rule, or computed; the taxable amount is floored at 0.00; and the total identity, asserted in every column. The graph (docs/ceg.md) becomes a decision table; constraints collapse impossible combinations (VAT-off makes the apply-to flags unobservable); each surviving column is exactly one test. Why: flag interactions are where ad-hoc testing under- or over-shoots; the graph makes every interaction explicit and proves the combination count.

Phase 4 - Structural analysis: CFG, V(G), and the coverage comparison

The control-flow graph of the module has 51 nodes (statement blocks and decision nodes, plus an ENTER and an EXIT node for each function) and 62 control edges, so V(G) = E - N + 2P = 19 (11 call edges drawn dashed). The graph is machine-verified, not hand-trusted: gen_cfg.py recounts the hand-encoded graph, cross-checks V(G) against an AST decision-point count, proves the basis-path set with a GF(2) rank argument, and actually executes each feasible basis path under sys.settrace - printing ALL CHECKS PASSED. The coverage comparison (docs/coverage-comparison.md) then shows line and branch coverage side by side for every suite, with the 212-test suite reaching 100% line AND branch - enforced by the coverage gate (fail_under = 100 in pyproject.toml; --cov-fail-under=100 in run_lab all), not just measured once.

Phase 5 - MC/DC (14 tests)

Modified Condition/Decision Coverage strengthens 'every branch taken' to 'every condition shown to independently affect the decision'. For each of the module's 14 reachable conditions (C1-C14 in docs/mcdc.md; every decision in pricing.py is a single condition - the module has no and/or, and the

three-way VAT selector in compute_totals (VAT off / override present / computed) is an if/elif chain of two such decisions), the truth tables in docs/mcdc.md exhibit the pair of runs where flipping ONLY that condition - with every other condition on the path held fixed - flips the outcome. Why it matters: branch coverage of A AND B can pass while B alone never matters in any test; MC/DC constructs precisely the rows that expose that - which is why DO-178B/C requires it for the most safety-critical (Level A) airborne software.

Phase 6 - Mutation testing: the cross-examination (3 hand-built + 176 mutmut + 226 cosmic-ray)

Green tests are my claim; mutants are the adversary that measures the claim. Hand-built first, one fault class each: the EQUIVALENT mutant (behavior-identical change - unkillable by any test at all, black-box or white-box - which is why, under this lab's score killed / (killed + survived), a 100% mutation score is impossible whenever such a mutant exists), the KILLED mutant (a fault my suites catch), and the SURVIVING mutant (for Req 6: fee_value <= 0 -> <= 4, which survives the chosen MC/DC test - the fee-type pair, which fixes fee_value=5 - but is killed by the full suite, where 8 BVA tests and 2 MC/DC tests fail; for Req 7: tool mutants that survive a single suite expose that technique's specific blind spots). Then the tools, mutmut (176 mutants) and cosmic-ray (226), each run against all four suites, with per-suite scores in docs/mutation-analysis.md - the comparison of tools is itself a delivered analysis point.

Phase 7 - Property-based testing with Hypothesis (8 properties)

Every earlier phase asserts on inputs I chose; this one states invariants that must hold for EVERY input and lets Hypothesis search the input space for a counterexample. Eight properties over randomly generated carts - idempotence and bounded drift of the money quantizer, conservation of the total, the taxable floor, non-negative VAT, two-decimal money shape, the fee base identity, and override-replaces-rate - run at 200 examples each in the default run and 10,000 each in the deep run (SLOW=1). Section 8 carries the invariants, the example budgets and the measured result.

4. Results at a glance

Deliverable	Result
Equivalence partitioning	52 tests + partition table
Boundary-value analysis	129 tests + boundary tables
Cause-effect graphing	17 tests from the decision table
MC/DC	14 tests with truth-table justification
Property-based testing (Hypothesis)	8 properties - 200 examples each in the default run, 10,000 each in the deep run
Total	220 lab tests, all green
Coverage on pricing.py	100% line + 100% branch, enforced by the coverage gate
Control flow	51 nodes / 62 edges / V(G)=19 - machine-verified
Mutation, hand-built	equivalent + killed + surviving, each explained
Mutation, tools	mutmut 176 + cosmic-ray 226, per-suite scores

5. The end-to-end walkthrough (what you show live)

Two commands, two minutes, two proofs:

- python -m pytest --cov --cov-branch -> 220 passed ... 100.00% line+branch (the functional + coverage claim, live)
- python docs/gen_cfg.py -> ALL CHECKS PASSED (the CFG/V(G)/basis-path claim, self-verified by the machine)

If time allows, the mutation demo (mutants/run_demo.py) runs all three hand-built mutants side by side - the strongest 60 seconds of the presentation: the audience watches the killed mutant turn tests red, the surviving mutant pass the single chosen MC/DC test (fee_value=5) and then die in the full suite (10 failures), and the equivalent mutant pass all 212 example-based tests because it changes nothing.

6. Expected questions - and the answers

Question	Answer
Why does an equivalent mutant exist?	It changes code without changing observable behavior; no test, black-box or white-box, can kill it. Under the score used here, killed / (killed + survived), its existence is why a 100% mutation score is impossible - an honest ceiling, not a failure.
Why BVA on top of EP?	Partitions prove every distinct behavior exists; boundaries are where implementations actually break (ties, off-by-one, truncation). EP is breadth; BVA is depth at the edges.
Is 100% coverage meaningful?	It is necessary, not sufficient: it proves every line and branch executed, not that faults would be detected. That insufficiency is precisely why MC/DC and mutation testing follow it - the strongest sentence in the lab.
Why is MC/DC stronger than branch coverage?	Branch coverage proves each branch was taken; MC/DC proves each CONDITION independently changes the outcome. A AND B can be branch-covered while B alone never mattered in any test.
Why was the spec written after the program?	The program pre-dates the lab (my own production port). Disclosed in the README; mitigated by writing the spec purely in black-box terms and deriving the EP, BVA and CEG suites from the spec, never the code; MC/DC, being structural, takes its list of conditions from the code and says so in docs/mcdc.md.
What does a surviving mutant mean?	In Req 6, mutant_surviving (fee_value <= 0 -> <= 4) survives the single chosen MC/DC test - the fee-type pair test_mcdc_c03, which uses fee_value=5 - because MC/DC freezes non-target inputs, but the full suite kills it (10 failures: 8 BVA tests and the C1/C2 MC/DC tests). In Req 7 (mutmut/cosmic-ray per-suite runs), survivors trace either to a single technique's blind spot (e.g. MC/DC holding default

	rounding, BVA never switching apply-to-fee on or calling compute_vat with VAT off) or to equivalent/runtime-dead mutations (like cosmic-ray mutating Decimal | None type annotations).

7. File map (what ships in this package)

Path	What it is
pricing.py	module under test
SPEC.md	the black-box specification
tests/test_ep.py|test_bva.py|test_ceg.py| test_mcdc.py	the four suites
tests/test_properties.py	the Hypothesis property suite (Phase 7)
docs/ep.md|bva.md|ceg.md|mcdc.md	derivation documents (tables + justifications)
docs/cfg.drawio + cfg.svg + cfg-analysis.md + gen_cfg.py	CFG, V(G), basis paths + the verifier
docs/coverage-comparison.md	per-suite line vs branch coverage
docs/mutation-analysis.md + mutants-manual.md + mutants/	mutation scores + the three hand-built mutants
REPORT.md	the assembled lab report
README.md	start here: setup, how to run everything, and the plain-language overview
AI_USAGE.md	the AI usage log: tool, purpose, prompt, result, edits and validation for each use
FUNCTIONS.md	quick reference: the four functions and their fault sites
PACKAGE.md	reading order and commands for this package
run_lab.ps1 + run_lab.sh	one-command runners (PowerShell / Bash): all, ep, bva, ceg, mcdc, properties, deep, coverage, cfg, mutants, clean
pyproject.toml + tests/conftest.py	pytest and coverage settings (the 100% gate) and the two Hypothesis profiles
TSS_Lab_Presentation.pptx	the slide deck
TSS_Lab_Documentation.docx + .pdf	this document (Word and PDF)

8. Phase 7 - Property-based testing (Hypothesis)

Phases 1 to 6 share one limitation: in every one of them I name the inputs. Equivalence partitioning picks one representative per class, boundary-value analysis picks the points either side of each edge, cause-effect graphing picks the 17 columns of the decision table, MC/DC picks the 14 pairs of runs (28 executions) that make each condition matter. Each is a finite, hand-chosen sample, so a fault that lives strictly between my chosen points is invisible to all of them. Phase 7 inverts the question: instead of naming inputs and asserting outputs, it states invariants that must hold for EVERY input and lets Hypothesis generate inputs looking for a counterexample. The suite is tests/test_properties.py - eight

properties over randomly generated carts, each one naming its invariant (Q1-Q2, T1-T4, F1, V1) in its docstring.

8.1 The invariants asserted

Cart inputs are generated as 2-decimal-place Decimals of either sign, with magnitudes bounded to 1,000,000 for cart amounts and 1,000 for rates and fee values; the quantizer properties Q1 and Q2 instead draw values with up to 6 decimal places, so that rounding actually happens. With these bounds the largest intermediate - the VAT product base * vat_rate_percent / 100, at most 120,000,000 - stays well inside Decimal's 28-significant-digit context and cannot raise InvalidOperation in place of a real assertion failure. Flags are plain booleans; rounding modes and fee types are drawn from the four and two legal values respectively. Negative draws are deliberate - they are what exercises the <= 0 guards and the taxable floor.

- Q1 - idempotence: quantize_money applied to an already-quantized result returns that result unchanged, for every value and in all four rounding modes.
- Q2 - bounded drift and sign preservation: quantization moves a value by at most one cent (|out - in| <= 0.01) and never flips its sign - positive inputs stay >= 0, negative inputs stay <= 0 - in all four modes.
- T1 - conservation: total == taxable_subtotal + shipping + fee + vat, for any amounts, any combination of the four on/off flags, either fee type, any rounding mode, with or without a VAT override. Money is neither created nor destroyed by the composition.
- T2 - the discount is bounded: taxable_subtotal is floored at 0.00 and never exceeds max(subtotal, 0.00). The upper bound is the CLAMPED subtotal, not the raw one, and that distinction was not mine - the naive form 'taxable_subtotal <= subtotal' is false by design, and Hypothesis falsified it at subtotal = -0.01, where the same floor lifts a negative subtotal to 0.00. The property suite corrected the property author.
- T3 - VAT is never negative, for any rate, any flag combination, override included.
- T4 - money shape: every field of the breakdown (subtotal, discount, taxable_subtotal, shipping, fee, vat, total) is quantized to exactly two decimal places - asserted on the Decimal exponent being -2, not on a string.
- F1 - fee base identity: the fee is charged on the TAXABLE subtotal, never on the pre-discount subtotal. This is the classic composition fault the suite exists to catch, and the expected fee is recomputed from the original inputs by an independent oracle quantizer - never read back out of a breakdown field, which would make the assertion circular.
- V1 - override semantics: an enabled VAT override REPLACES the rate-based VAT rather than adding to it. Asserted twice over: the reported VAT equals the quantized override when that override is positive and 0.00 otherwise, and re-running the same cart with a different vat_rate_percent must not move the reported VAT - so, on every generated cart, changing the rate had no effect once an override was present.

8.2 Why properties catch faults the example-based suites miss

- Unnamed inputs. EP, BVA, CEG and MC/DC assert on points I chose; a property asserts on points Hypothesis chooses, including the ones I did not think to write down. A fault that only surfaces at some arbitrary interior value has nowhere left to hide.

- Relations instead of literals. T1 constrains the RELATIONSHIP between output fields; F1 compares the fee with a value recomputed from the original inputs by an independent oracle, instead of a hand-computed number in a table. If an expected value in a table was itself derived from a misreading of the spec, a mutant that shares that misreading still passes that example test; F1 can still catch it, because its oracle is computed independently of the code under test (unless the property's author made the same misreading). T1 alone would not: conservation holds whatever fee base the code uses.
- Automatic shrinking. When a property fails, Hypothesis minimises the counterexample before reporting it, so what arrives is the smallest input that breaks the invariant - a boundary case found rather than guessed. T2 is the recorded instance in this lab: the first version of the property was wrong, and the shrunk counterexample (subtotal = -0.01) named the exact reason.
- Complementary, not a replacement. A property says 'this relation always holds'; it does not say 'this input produces this exact output'. The example-based suites remain the ones that pin the SPEC's worked examples, and they are what the Phase 6 mutation analysis was run against. Phase 7 adds a different kind of evidence - it does not retire the earlier phases, and no mutation score is claimed for it.

8.3 Example budgets - the CI-safe default and the deep run

Hypothesis costs wall-clock time per generated example, so the budget is parameterised rather than fixed - the suite must not slow the CI matrix, and the deep pass must still be reachable on demand:

- Default lane (the profile named "ci"): max_examples = 200 per property. This is the profile that runs in the ordinary suite.
- Deep local pass: SLOW=1 in the environment raises max_examples to 10,000 per property.
- Two switches select the 10,000-example deep lane, and both work: setting SLOW=1 in the environment, or passing --hypothesis-profile slow to pytest. tests/conftest.py registers both profiles - "ci" (max_examples = 200) and "slow" (max_examples = 10,000) - and loads "slow" when SLOW=1 is set, "ci" otherwise; when --hypothesis-profile is given on the command line, the named profile is used instead.

8.4 The measured result

Both lanes were run on the final file, and both are green:

- Default lane (profile "ci"), 200 examples per property: all 220 tests collected here (212 example-based + 8 Hypothesis properties) passed, exit 0. Coverage holds at 100.00% line and 100.00% branch on pricing.py (57 statements, 22 branches) - "Required test coverage of 100.0% reached." The type check (basedpyright, standard mode) reports 0 errors.
- Deep lane, SLOW=1 and 10,000 examples per property: 8 passed, exit 0 (130.52 s on the original run; wall time varies with machine load). The Hypothesis statistics are the part worth reading - all eight properties report "10000 passing, 0 failing, and 0 invalid test cases" and "Stopped because settings.max_examples=10000". Every property therefore spent its entire budget instead of stopping early, and not one generated case was discarded as invalid, so the 10,000 is a real 10,000.

- No counterexample survives either lane: the eight invariants hold across the generated input space at both budgets. The suite adds 8 tests to the lab's 212, for 220 collected in this package's tests/ folder (212 example-based + 8 Hypothesis properties).

9. Contribution declaration and AI usage

The assignment asks every team member to declare which of points 1-7 they contributed to and their own percentage, with the percentages adding up to 100%. This is an individual submission:

Member	Points contributed to and own percentage
Andrei Mihai Visalon	points 1-7 (all) - 100%

The assignment also allows AI tools and asks, for every significant use, for the tool, the purpose, the prompt or request, the result, the changes made by the team, and how the result was verified and validated. AI_USAGE.md in this package records all 13 significant uses in exactly those six fields; the table below summarises them.

Use (assignment point)	Tool, and how the result was verified
1. Program port and project skeleton (base)	ZCode agent (GLM model). Behaviour-identical port; every suite runs green against it.
2. Specification, SPEC.md (base)	ZCode agent (GLM model). Every SPEC rule is exercised by at least one test; the combined run reaches 100% branch coverage.
3. EP suite and derivation (point 1a)	ZCode agent (GLM model). 52 passed; expected values re-derived by hand from the SPEC rules.
4. BVA suite and derivation (point 1b)	ZCode agent (GLM model). 129 passed; boundary expectations hand-checked against decimal rounding.
5. CEG suite and derivation (point 1c)	ZCode agent (GLM model). 17 passed; every decision-table column maps to exactly one test.
6. MC/DC suite and derivation (point 4)	ZCode agent (GLM model). 14 passed; conditions inventoried from pricing.py, expected values hand-derived from the code paths.
7. CFG artifacts and verifier (point 3)	ZCode agent (GLM model). docs/gen_cfg.py runs 69 checks: V(G) three ways, GF(2) rank, 18 executable basis paths traced.
8. Coverage comparison (point 2)	ZCode agent. Exact commands embedded in the document; re-measured on 05.11.2026 with identical figures.
9. Hand-built mutants and demo (points 5-6)	ZCode agent (GLM model). mutants/run_demo.py exits 0 only if every claim matches - it does (re-run 05.11.2026).
10. Mutation analysis with two tools (point 7)	ZCode agent (GLM model) running mutmut 3.8.0 and cosmic-ray 8.7.0; both tools re-run per suite on 05.11.2026 with the recorded protocol: mutmut reproduced its scores exactly; cosmic-ray reproduced its scores exactly.

11. Round-1 critic fixes	ZCode agent (GLM model). Full suite, 100% gate and mutant demo re-run after the fixes.
12. Property-based tests (bonus)	ZCode agent, completed by Claude Code (Anthropic). 220 passed at 200 examples per property and 8 passed at 10,000; property T2 was corrected after Hypothesis falsified it.
13. Documentation audit, corrections and re-verification (all points)	Claude Code (Anthropic), plus the automated Sentry Seer review of the repository pull request (each of its findings was reproduced by running the code, then fixed). run_lab all re-run on the final package (220 passed, 100% line and branch, 69 checks, all mutant claims match), also on Python 3.10 and 3.11 and in Windows PowerShell 5.1; each correction checked against the code.

At the presentation the author explains each of these uses and how its result was checked.

Prepared as one package with the slide deck - Andrei Mihai Visalon